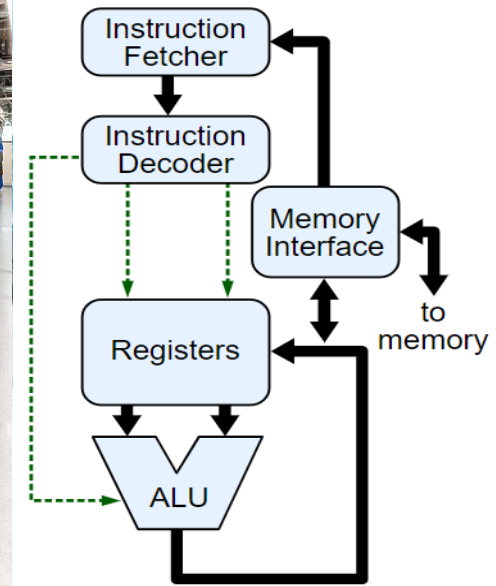
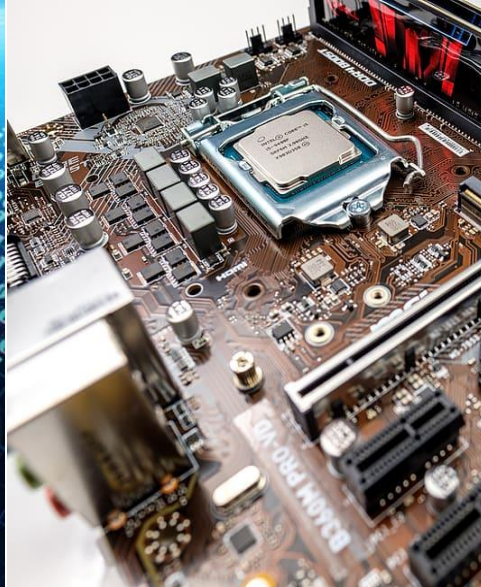
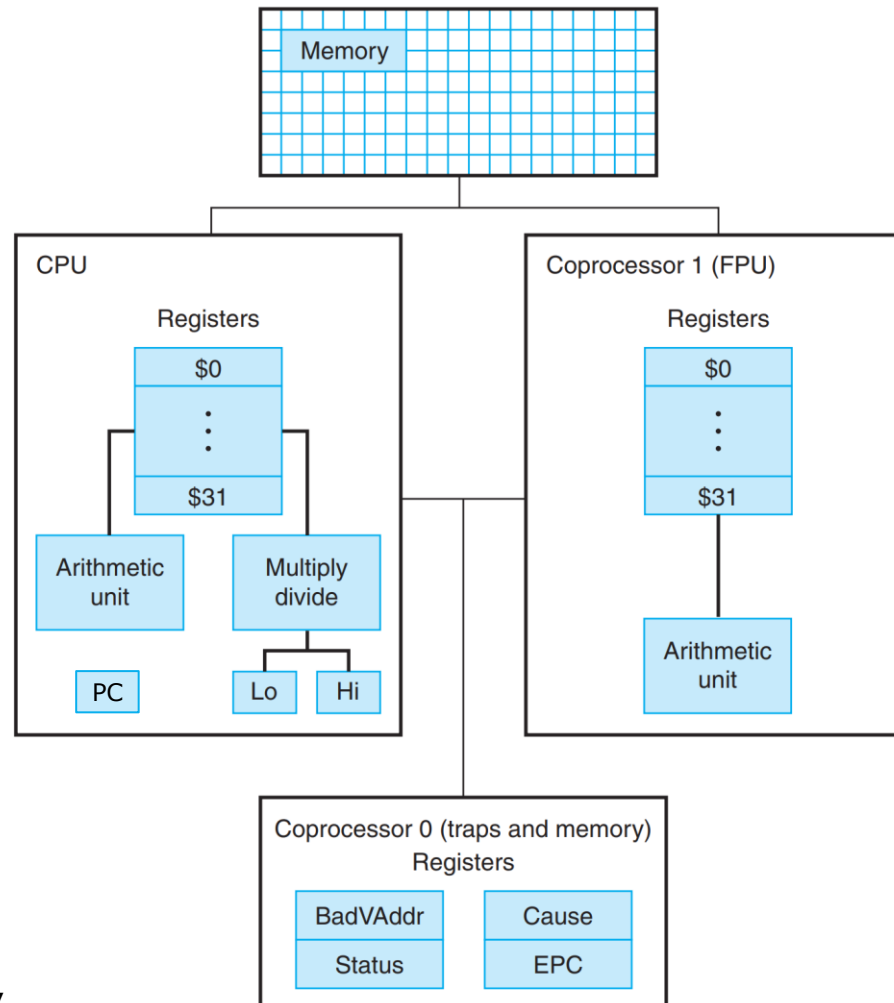




Recap

- ▶ We covered the last instruction type (*J-ump*) and discussed how different constructs (loops, if-else, functions) are implemented in assembly.
- ▶ (*Pre-recorded*) We saw how C code is compiled into MIPS assembly and the resulting file.
- ▶ Background reading: Appendix A.7.

The MIPS2000 coprocessors



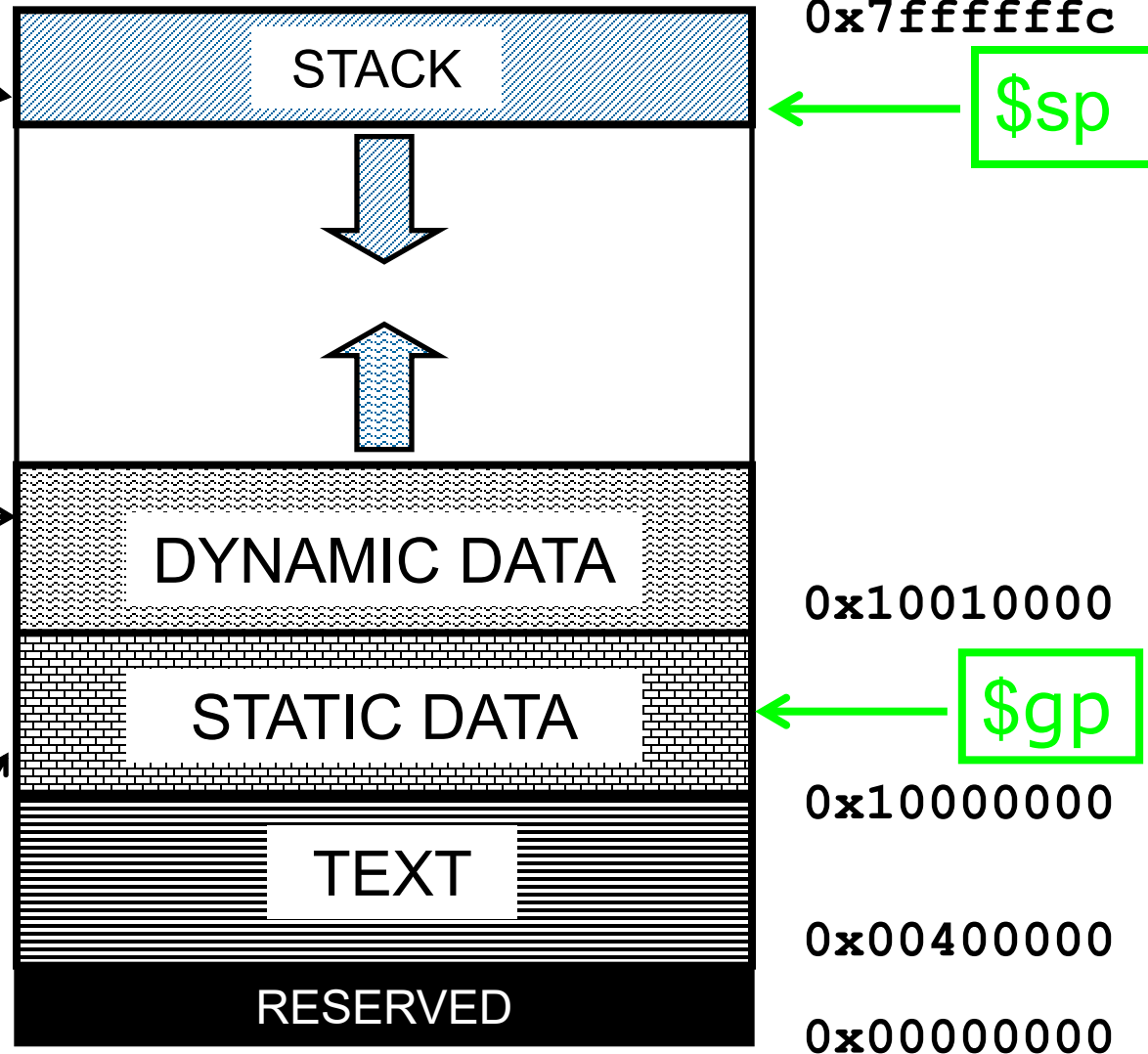
Patterson & Hennessy

Variables with different scopes / purposes are allocated in different regions of memory

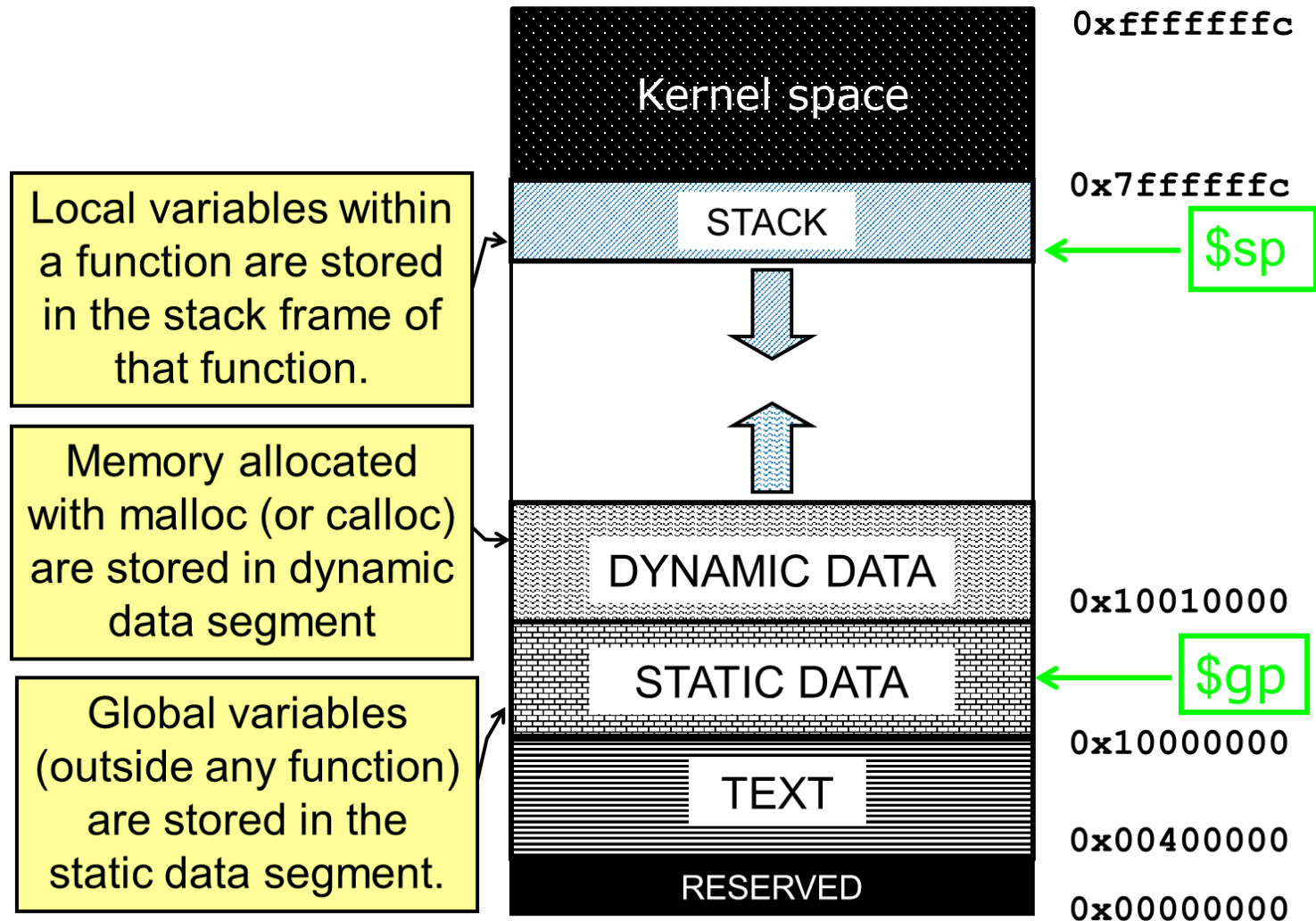
Local variables within a function are stored in the stack frame of that function.

Memory allocated with malloc (or calloc) are stored in dynamic data segment

Global variables (outside any function) are stored in the static data segment.



Variables with different scopes / purposes are allocated in different regions of memory



The kernel handles all exceptions

- ▶ That is, the PC changes to address stored in the kernel space.
 - It can store the state to return to the program if needed.
 - It can run instructions that are not allowed in user space (e.g., accessing 0xFFC0FFEE or halting the machine).
- ▶ We discussed overflow exceptions.
- ▶ There are many different types of exceptions.

What happens on exceptions?

- ▶ The *exception code* is loaded (among other things) to coprocessor 0's `cause` register.
- ▶ The current PC value is stored in coprocessor 0's `epc` register.
- ▶ The PC is changed to 0x80000080, the location of the *exception handler* code.
- ▶ `$k0, $k1` may serve as *scratch* registers without being restored.

List of exceptions

Number	Name	Cause of exception
0	Int	interrupt (hardware)
4	AdEL	address error exception (load or instruction fetch)
5	AdES	address error exception (store)
6	IBE	bus error on instruction fetch
7	DBE	bus error on data load or store
8	Sys	syscall exception
9	Bp	breakpoint exception
10	RI	reserved instruction exception
11	CpU	coprocessor unimplemented
12	Ov	arithmetic overflow exception
13	Tr	trap
15	FPE	floating point

The `syscall` instruction

- ▶ A native CPU instruction with encoding `0x0000000c`.
 - Technically, an *R*-type instruction where `opcode=rs=rt=rd=0`.
- ▶ Includes a set of operating system services.
- ▶ We indicate the system call number in `$v0`.
- ▶ Otherwise follows the convention of taking input parameters in `$a0–$a3`. (*)

Examples of syscalls available with MIPS / MARS

Service	System Call (loaded into \$v0)	Arguments	Results
print_int	1	\$a0 = integer	
print_float	2	\$f12 = float	
print_double	3	\$f12 = double	
print_string	4	\$a0 = string	
read_int	5		integer (in \$v0)
read_float	6		float (in \$f0)
read_double	7		double (in \$f0)
read_string	8	\$a0 = buffer \$a1 = length	
exit	10		
read_char	12		char (in \$v0)

Example – Reading two numbers and writing their sum

```
# Interactive calculator - add 2 numbers together

.data
str1: .asciiz "\nEnter first value: " # Note initial newline.
str2: .asciiz "Enter second value: "
str3: .asciiz "Sum is equal to "

.text
.globl main
main: addi $v0, $0, 4      # Set system call code (print_string)
     la    $a0, str1      # Load address of string into $a0.
     syscall              # Print string.

     addi $v0, $0, 5      # Set system call code (read_int)
     syscall              # Read integer into $v0.
     add $t1, $0, $v0     # Store integer in temporary register.
```

Example – Reading two numbers and writing their sum (continued)

```
addi $v0, $0, 4      # Set system call code (print_string)
la    $a0, str2       # Load address of string into $a0.
syscall              # Print string.

addi $v0, $0, 5      # Set system call code (read_int)
syscall              # Read integer into $v0.
add $t1, $t1, $v0     # Add newly entered value to previous.

addi $v0, $0, 4      # Set system call code (print_string)
la    $a0, str3       # Load address of string into $a0.
syscall              # Print string.

addi $v0, $0, 1      # Set system call code (print_int)
add $a0, $0, $t1      # Print result.
syscall              # Call the system service.

li $v0 10             # Exit program using system call 10.
syscall              # syscall 10 (exit)
```

Summary

- We discussed the MIPS coprocessors.
- We looked into exception handling and the kernel memory segment.
- Overviewed several useful system calls.
- For those interested: in [COMP0019](#) you'll study the kernel, virtual addresses, and system calls in much more depth.